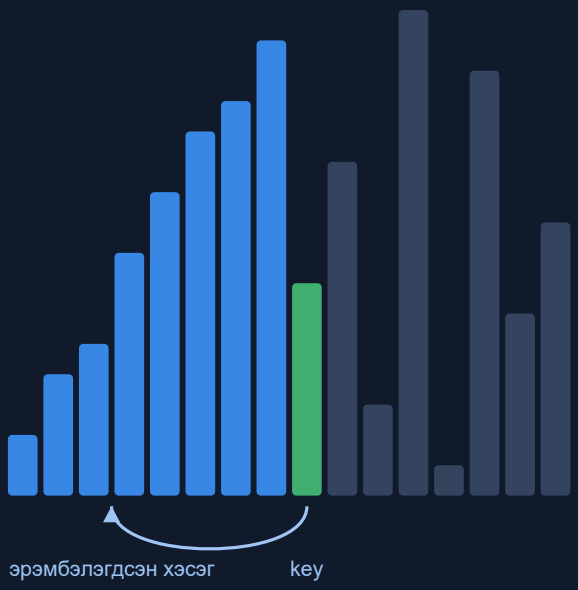
АЛГОРИТМЫН ҮНДЭС · PYTHON · ЭРЭМБЭЛЭЛТ 2/3

Insertion sort

Оруулан эрэмбэлэлт: хөзөр гартаа эрэмбэлдэг шиг



Түвшин: Ойлголт + хэрэгжүүлэлт | **Жишээ:** Python 3 | **Урьдчилсан мэдлэг:** Selection sort, while давталт, Big O

Энэ хичээлийг уншсаны дараа та:

- Insertion sort-ийн санааг **хөзрийн** жишээгээр ойлгоно.
- «Шилжүүлэх» алхмыг зураг, хүснэгтээр **мөрдөж**, insertion sort-ийг өөрөө бичнэ.
- Яагаад эрэмбэлэгдсэн жагсаалтад **O(n)**, урвуу жагсаалтад **O(n²)** болохыг тайлбарлана.
- **Тогтвортой** (stable) эрэмбэлэлт гэж юу болохыг мэднэ.
- Selection sort-тэй харьцуулж, аль нь хэзээ илүүг хэлнэ.

Агуулга

01 Санаа
02 Код
03 Гараар мөрдөх
04 Түгээмэл алдаа ба тест
05 Хэр хурдан бэ?
06 Тогтвортой эрэмбэлэлт
07 Selection sort-тэй харьцуулах
08 Дасгал
09 Хураангуй

Нэг өгүүлбэрээр

Insertion sort (оруулан эрэмбэлэлт): дараагийн элементийг ав, зүүн талын эрэмбэлэгдсэн хэсэгт түүнээс их элементүүдийг баруун тийш шилжүүлж, чөлөөлөгдсөн байранд нь **оруул**.

01 Санаа

> **Өмнөх хичээлээс (Эрэмбэлэлт 1/3: Selection sort)**
> Жагсаалтыг эрэмбэлэгдсэн (зүүн) ба эрэмбэлэгдээгүй (баруун) хэсэгт хувааж, баруунаас хамгийн багыг сонгож зүүн хэсэгт залгадаг. Үргэлж $n(n-1)/2$ харьцуулалт — $O(n^2)$. Insertion sort ч мөн хоёр хэсэгт хуваана, гэхдээ ажлаа эсрэгээр нь хийдэг.

Зүйрлэл: хөзөр тоглохдоо ширээнээс хөзрөө нэг нэгээр нь авч, гартаа аль хэдийн эрэмбэлсэн хөзрүүдийнхээ дунд **зөв байранд нь** оруулдаг. Шинэ хөзрөөс том хөзрүүд баруун тийш нэг зай шилжиж зай тавьж өгнө.

Ялгааг анзаар:

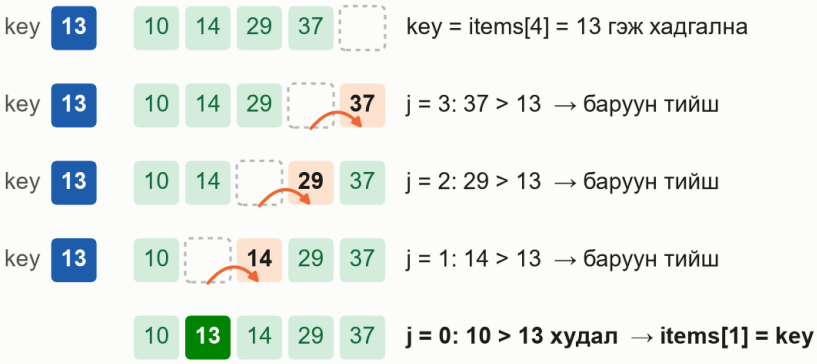
- **Selection sort:** баруун хэсгээс **аль** элементийг авахаа сонгоход хүч зарцуулна (хамгийн багыг хайна); зүүн хэсэгт залгах нь хялбар.
- **Insertion sort:** баруун хэсгийн **дараагийн** элементийг шууд авна; харин зүүн хэсэгт **хаана** оруулахаа олоход хүч зарцуулна.
- Тиймээс insertion sort-ийн зүүн хэсэг эрэмбэлэгдсэн ч элементүүд нь **эцсийн байрандаа биш** — дараа нь ирэх бага элемент тэднийг баруун тийш түлхэж болно.



Зураг 1. [29, 10, 14, 37, 13]-ийг insertion sort-оор эрэмбэлэх нь. Ногоон — эрэмбэлэгдсэн зүүн хэсэг, хар хөх — оруулах элемент (key), цайвар хөх — хараахан ороогүй хэсэг, улбар шар сум — key орох байр.

02 Код

Кодын гол хэсэг бол нэг эргэлт: key-г гаргаж аваад, түүнээс их элементүүдийг нэг нэгээр баруун тийш шилжүүлж, гарсан «нүх»-нд key-г тавих. Сүүлийн эргэлтийг ойроос харъя:



Зураг 2. Нэг эргэлтийг ойроос (i = 4, key = 13). Тасархай — чөлөөтэй «нүх», улбар шар — дөнгөж баруун тийш шилжсэн элемент. Нүх зүүн тийш явсаар key-ийн зөв байранд хүрнэ.

Алхам алхмаар бичье

Алхам 1. 0-р элементийг ганцаараа «эрэмбэлэгдсэн» гэж үзээд 1-р индексээс эхэлнэ: `for i in range(1, len(items)):`

Алхам 2. Оруулах элементийг **хадгална** — шилжүүлэлтэд дарагдаж алга болох тул: `key = items[i]`

Алхам 3. Зүүн тийш явж, key-ээс их элемент бүрийг нэг байр баруун тийш шилжүүлнэ: `while j >= 0 and items[j] > key:`

Алхам 4. Давталт зогсоход `j` нь key-ээс их биш элемент дээр (эсвэл −1) байна. key-ийн байр бол түүний **дараагийнх**: `items[j + 1] = key`

```python
def insertion_sort(items):
    for i in range(1, len(items)):          # 1. 0-р элемент ганцаараа эрэмбэлэгдсэн
        key = items[i]                      # 2. оруулах элементийг хадгална
        j = i - 1
        while j >= 0 and items[j] > key:    # 3. key-ээс их бол…
            items[j + 1] = items[j]         #    …баруун тийш шилжүүлнэ
            j -= 1
        items[j + 1] = key                  # 4. чөлөөлөгдсөн байранд оруулна

nums = [29, 10, 14, 37, 13]
insertion_sort(nums)
print(nums)                                 # [10, 13, 14, 29, 37]
```

> **Яагаад for биш while вэ?**
> Дотоод давталт хэдэн удаа эргэхийг урьдчилж мэдэхгүй: key-ээс их биш элемент тааралдмагц, эсвэл жагсаалтын эхэнд хүрмэгц зогсоно. Ийм хоёр нөхцөлтэй давталтад `while` тохиромжтой. `j >= 0` шалгалт эхэнд байх ёстой — эс бөгөөс j = −1 үед `items[-1]` буюу **сүүлийн** элементийг дэмий уншина (Хайлт 1/2-ийн анхааруулгыг сана); C, Java зэрэг хэлэнд энэ нь шууд алдаа.

03 Гараар мөрдөх

i	key	Эргэлтийн өмнө	Шилжилт	Харьц.	Эргэлтийн дараа
1	10	`[29, 10, 14, 37, 13]`	1	1	`[10, 29, 14, 37, 13]`
2	14	`[10, 29, 14, 37, 13]`	1	2	`[10, 14, 29, 37, 13]`
3	37	`[10, 14, 29, 37, 13]`	0	1	`[10, 14, 29, 37, 13]`
4	13	`[10, 14, 29, 37, 13]`	3	4	`[10, 13, 14, 29, 37]`
		Нийт	**5**	**8**	

i = 3 үед key = 37 зүүн хөрш 29-өөсөө их тул **ганц** харьцуулаад байрандаа үлдлээ. Insertion sort-ийн хурдны нууц үүнд бий (5-р бүлэг).

Дотоод давталт: `i = 4`, `key = 13`

j	items[j]	items[j] > 13 ?	Үйлдэл	Жагсаалт (жинхэнэ төлөв)
3	37	**True**	`items[4] = 37`	`[10, 14, 29, 37, 37]`
2	29	**True**	`items[3] = 29`	`[10, 14, 29, 29, 37]`
1	14	**True**	`items[2] = 14`	`[10, 14, 14, 29, 37]`
0	10	False	давталт зогсоно	`[10, 14, 14, 29, 37]`
—			`items[1] = 13`	`[10, 13, 14, 29, 37]`

> **Анхаар: түр зуурын давхардал**
> Шилжүүлэх явцад жагсаалтад нэг утга хоёр удаа харагдана (`[10, 14, 14, 29, 37]`). Энэ алдаа биш: 13 нь `key` хувьсагчид аюулгүй хадгалагдсан бөгөөд эцэст нь давхардсан байрын нэгэнд бичигдэнэ. Зураг 2-ын «нүх» бол яг энэ байр.

04 Түгээмэл алдаа ба тест

Алдаа 1 · key-г хадгалахгүй байх

БУРУУ

```
j = i - 1
while j >= 0 and items[j] > items[i]:
    items[j + 1] = items[j]
    j -= 1
items[j + 1] = items[i]
```

ЗӨВ

```
key = items[i]
j = i - 1
while j >= 0 and items[j] > key:
    items[j + 1] = items[j]
    j -= 1
items[j + 1] = key
```

Юу болох вэ: `[29, 10, 14]` → `[29, 29, 29]` — 10, 14 **алга болсон**!

Яагаад: Эхний шилжилт `items[i]`-г дарж бичнэ (`j + 1 == i`). Үүний дараа `items[i]` нь оруулах гэсэн элемент биш, шилжсэн элемент болсон тул оруулах утга мөнхөд алдагдана.

Алдаа 2 · items[j + 1]-ийн оронд items[j]-д оруулах

БУРУУ

```
while j >= 0 and items[j] > key:
    items[j + 1] = items[j]
    j -= 1
items[j] = key
```

ЗӨВ

```
while j >= 0 and items[j] > key:
    items[j + 1] = items[j]
    j -= 1
items[j + 1] = key
```

Юу болох вэ: `[29, 10, 14]` → `[29, 29, 10]`.

Яагаад: Давталт зогсоход `items[j]` нь key-ээс их **биш** элемент — түүнийг дарах ёсгүй. Хамгийн аюултай нь j = −1 үед `items[-1] = key` алдаагүй ажиллаж **сүүлийн** элементийг чимээгүй дарна.

Алдаа 3 · while-ийн оронд if

БУРУУ

```
if j >= 0 and items[j] > key:
    items[j + 1] = items[j]
    j -= 1
items[j + 1] = key
```

ЗӨВ

```
while j >= 0 and items[j] > key:
    items[j + 1] = items[j]
    j -= 1
items[j + 1] = key
```

Юу болох вэ: `[3, 2, 1]` → `[2, 1, 3]`.

Яагаад: key ердөө нэг байр л зүүн тийш явна. 1 нь хоёр байр явах ёстой байсан. Хэдэн удаа шилжихийг урьдчилж мэдэхгүй тул давталт хэрэгтэй.

Тест

Selection sort-ийн тестийг (1/3, 7-р бүлэг) функцийн нэрийг л сольж ашиглана: `sorted()`-тэй харьцуулсан санамсаргүй тест + захын тохиолдлууд.

Зөв хувилбар **10 000** санамсаргүй тестийг бүгдийг давсан. Алдаатай хувилбарууд:

Хувилбар	1 000 тестээс бүтэлгүйтсэн	Хамгийн богино жишээ	Алдааны төрөл
Алдаа 1: key хадгалаагүй	**763**	`[1, 0] → [1, 1]`	буруу хариу
Алдаа 2: `items[j] = key`	**826**	`[0, 1] → [1, 1]`	буруу хариу
Алдаа 3: `if` (while биш)	**640**	`[2, 6, 0] → [2, 0, 6]`	буруу хариу
Зөв `insertion_sort`	**0**	—	—

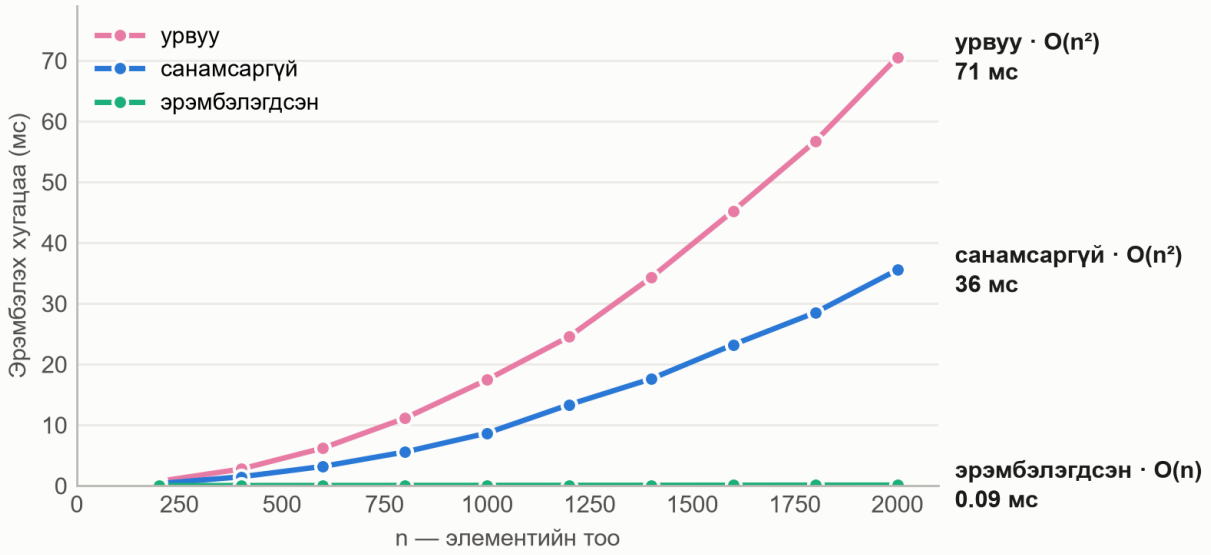
05 Хэр хурдан бэ?

Selection sort-оос ялгаатай нь insertion sort-ийн ажил **өгөгдлөөс** хамаарна:

Тохиолдол	Оролтын жишээ	Харьцуулалт	Шилжилт	Big O
Хамгийн сайн	эрэмбэлэгдсэн `[1, 2, 3, 4, 5]`	n − 1	0	**O(n)**
Дундаж	санамсаргүй	≈ n² / 4	≈ n² / 4	O(n²)
Хамгийн муу	урвуу `[5, 4, 3, 2, 1]`	n(n − 1) / 2	n(n − 1) / 2	O(n²)

Эрэмбэлэгдсэн жагсаалтад key бүр зүүн хөрштэйгээ **ганц** удаа харьцуулагдаад шууд зогсоно. Урвуу жагсаалтад key бүр эрэмбэлэгдсэн хэсгийг бүхэлд нь туулж эхэнд хүрнэ. 1 000 элементтэй жагсаалтууд дээр тоолуулбал:

Ролт	Харьцуулалт	Шилжилт
эрэмбэлэгдсэн	999	0
бараг эрэмбэлэгдсэн (10 хос ойрхон элемент сольсон)	1 087	88
санамсаргүй	249 709	248 714
урвуу	499 500	499 500



Зураг 3. insertion_sort-ийн хугацаа гурван төрлийн оролт дээр. Энэ PDF-ийг үүсгэх үед бодитоор хэмжсэн. Эрэмбэлэгдсэн оролтын шугам тэг дээр хэвтэж байна — O(n).

n = 2 000 үед: эрэмбэлэгдсэн — **0.09 мс**, санамсаргүй — **36 мс**, урвуу — **71 мс**. Ижил функц, ижил n — гэтэл **808 дахин** зөрүүтэй.

> **Шилжилтийн тоо = «буруу хос»-ын тоо**
>
> Жагсаалтад өмнө байгаа нь хойно байгаагаасаа их байх хосыг **инверс** (inversion) гэдэг. Нэг шилжилт яг нэг инверсийг арилгадаг тул шилжилтийн тоо = инверсийн тоо. [29, 10, 14, 37, 13]-д 5 инверс бий: (29, 10), (29, 14), (29, 13), (14, 13), (37, 13) — 3-р бүлгийн хүснэгтийн нийт шилжилт ч яг 5. Бараг эрэмбэлэгдсэн жагсаалтад инверс цөөн тул insertion sort бараг O(n).

06 Тогтвортой эрэмбэлэлт

Ижил түлхүүртэй (жишээ нь ижил оноотой) элементүүд эрэмбэлсний дараа **анхны дарааллаа хадгалвал** тэр эрэмбэлэлтийг **тогтвортой** (stable) гэнэ.

Сурагчдыг оноогоор нь өсөхөөр эрэмбэлье. Бат, Дорж хоёр ижил 85 оноотой бөгөөд анхны жагсаалтад Бат түрүүлж байна. Харьцуулалтыг `items[j][1]` (оноо) болгож өөрчилсөн хоёр алгоритмын үр дүн:

	Үр дүн	Бат Доржоос өмнө үү?
Анхны жагсаалт	Бат 85, Дорж 85, Сараа 70	Тийм
Selection sort	Сараа 70, Дорж 85, Бат 85	**Үгүй — солигдсон**
Insertion sort	Сараа 70, Бат 85, Дорж 85	**Тийм — хадгалагдсан**

- **Selection sort холын зайд солидог.** Сараа 0-р байр руу орохдоо тэнд байсан Батыг 2-р байр руу — Доржийн ард шидэв. Ийм холын солилт ижил элементүүдийн дарааллыг эвддэг.
- **Insertion sort хөршөө л шилжүүлдэг.** key зөвхөн өөрөөсөө **хатуу их** (`>`) элементийн хажуугаар өнгөрдөг тул ижил оноотойгоо хэзээ ч гүйцэж түрүүлэхгүй.

> **`>`-ийн оронд `>=` бичвэл**
> Insertion sort зөв эрэмбэлсээр байгаа ч **тогтвортой байхаа болино**: Сараа 70, Дорж 85, Бат 85. Нэмээд ижил элементүүдийг дэмий шилжүүлж удаашрна.

> **Хэзээ хэрэгтэй вэ?**
> Олон шатлалтай эрэмбэлэлт: эхлээд нэрээр, дараа нь оноогоор эрэмбэлэхэд тогтвортой алгоритм ижил оноотой сурагчдыг **нэрийн дарааллаар** нь үлдээнэ. Python-ий `sorted()` ба `list.sort()` тогтвортой.

07 Selection sort-тэй харьцуулах

Шинж	■ Selection sort	■ Insertion sort
Санаа	Хамгийн багыг сонгож урд нь тавина	Дараагийнхыг авч зөв байранд оруулна
Харьцуулалт (эрэмбэлэгдсэн)	n(n − 1) / 2	**n − 1**
Харьцуулалт (урвуу)	n(n − 1) / 2	n(n − 1) / 2
Бичих үйлдэл	**≤ n − 1 солилт**	≤ n(n − 1) / 2 шилжилт
Цаг (сайн / муу)	O(n²) / O(n²)	**O(n)** / O(n²)
Санах ой	O(1)	O(1)
Тогтвортой	Үгүй	**Тийм**

n = 2 000 элемент дээр хэмжсэн хугацаа:

Оролт	selection_sort	insertion_sort
эрэмбэлэгдсэн	32 мс	**87 мкс**
санамсаргүй	32 мс	36 мс
урвуу	35 мс	71 мс

Хэмжилтээс: санамсаргүй өгөгдөлд хоёулаа **ойролцоо**, урвуу өгөгдөлд insertion sort **2.0 дахин удаан** (шилжилт олон), харин эрэмбэлэгдсэн өгөгдөлд insertion sort **369 дахин хурдан**.

Бодит амьдрал дээрх өгөгдөл ихэвчлэн **бараг эрэмбэлэгдсэн** байдаг (эрэмбэлсэн жагсаалтад хэдэн шинэ мөр нэмэгдсэн гэх мэт), мөн insertion sort тогтвортой — тиймээс практикт ихэвчлэн insertion sort-ийг сонгодог. Python-ий `sorted()` хүртэл жижиг хэсгүүдийг (32–64 элемент) insertion sort-ийн хувилбараар эрэмбэлдэг. Selection sort-ийн давуу тал — бичих үйлдэл цөөн.

> **Дараагийн алхам: Merge sort**
> Хоёулаа O(n²) — сая элементэд хэдэн цаг. Эрэмбэлэгдсэн хоёр жагсаалтыг нэгтгэх нь маш хурдан гэдгийг ашиглан **O(n log n)** хүрч болно. Энэ бол «Эрэмбэлэлт 3/3: Merge sort» хичээлийн сэдэв.

08 Дасгал

Эхлээд өөрөө бодож, кодоо тестлээрэй. Хариуг дараагийн хуудаснаас шалга.

Дасгал 1 · Гараар мөрдөх

`insertion_sort([4, 3, 5, 1, 2])`: эргэлт бүрийн key, шилжилтийн тоо, эргэлтийн дараах жагсаалтыг бич.

Дасгал 2 · Буурахаар

`insertion_sort_desc(items)` бич: жагсаалтыг **буурахаар** эрэмбэлнэ.

```
nums = [3, 8, 1, 9, 4]
insertion_sort_desc(nums)
print(nums)                # [9, 8, 4, 3, 1]
```

Дасгал 3 · Шилжилт тоол

Код ажиллуулалгүйгээр: `[1, 2, 3, 4, 5]`, `[5, 4, 3, 2, 1]`, `[2, 1, 4, 3, 6, 5]` тус бүрд хэдэн шилжилт хийх вэ? (Зөвлөгөө: инверс тоол.)

Дасгал 4 · Эрэмбэлэгдсэн жагсаалтад нэмэх

`insert_sorted(items, x)`: эрэмбэлэгдсэн `items`-д x-ийг дарааллыг нь эвдэлгүй байрандаа нэмнэ. Insertion sort-ийн нэг эргэлт л хангалттай. Цагийн төвөгшил нь хэд вэ?

```
nums = [10, 14, 29, 37]
insert_sorted(nums, 13)
print(nums)                # [10, 13, 14, 29, 37]
```

Дасгал 5 · Тогтвортой юу?

`[('Ану', 3), ('Бат', 1), ('Цэцэг', 3), ('Дорж', 1)]`-ийг тоогоор нь insertion sort-оор эрэмбэлэв. `>` хэрэглэвэл ямар дараалал гарах вэ? `>=` хэрэглэвэл?

Дасгалын хариу

Бүх хариуг энэ PDF-ийг үүсгэх үед жишээн дээр нь ажиллуулж шалгасан.

1. Гараар мөрдөх — нийт 7 шилжилт

i	key	Шилжилт	Эргэлтийн дараа
1	3	1	`[3, 4, 5, 1, 2]`
2	5	0	`[3, 4, 5, 1, 2]`
3	1	3	`[1, 3, 4, 5, 2]`
4	2	3	`[1, 2, 3, 4, 5]`

2. Буурахаар — ганц тэмдэгт

```
def insertion_sort_desc(items):
    for i in range(1, len(items)):
        key = items[i]
        j = i - 1
        while j >= 0 and items[j] < key:    # >  →  <
            items[j + 1] = items[j]
            j -= 1
        items[j + 1] = key
```

3. Шилжилт тоол

`[1, 2, 3, 4, 5]` — **0** (инверсгүй). `[5, 4, 3, 2, 1]` — **10** (хос бүр инверс: 5 · 4 / 2). `[2, 1, 4, 3, 6, 5]` — **3** ((2, 1), (4, 3), (6, 5)).

4. Эрэмбэлэгдсэн жагсаалтад нэмэх — O(n)

```
def insert_sorted(items, x):
    items.append(x)                    # төгсгөлд нэг зай гаргана
    j = len(items) - 2                 # x-ийн өмнөх элемент
    while j >= 0 and items[j] > x:
        items[j + 1] = items[j]
        j -= 1
    items[j + 1] = x
```

Хамгийн муу үед бүх элементийг шилжүүлнэ — O(n). Python-д бэлэн нь: `bisect.insort(items, x)` (Хайлт 2/2) — байрыг O(log n)-ээр олдог ч шилжүүлэлт нь мөн O(n).

5. Тогтвортой юу?

`>`: `[('Бат', 1), ('Дорж', 1), ('Ану', 3), ('Цэцэг', 3)]` — Бат Доржоос, Ану Цэцэгээс өмнө хэвээр: **тогтвортой**. `>=`: `[('Дорж', 1), ('Бат', 1), ('Цэцэг', 3), ('Ану', 3)]` — ижил тоотой хосууд байраа сольсон: **тогтворгүй**.

09 Хураангуй

Загвар — цээжлэх биш, ойлгож бич

■ Insertion sort

```
def insertion_sort(items):
    for i in range(1, len(items)):
        key = items[i]
        j = i - 1
        while j >= 0 and items[j] > key:
            items[j + 1] = items[j]
            j -= 1
        items[j + 1] = key
```

■ Гол тоонууд

Сайн (эрэмбэлэгдсэн)	**n − 1 харьцуулалт · O(n)**
Муу (урвуу)	**n(n − 1) / 2 · O(n²)**
Шилжилт	**= инверсийн тоо**
Санах ой	**O(1) — байрандаа**
Тогтвортой юу?	**Тийм (`>` үед)**

Insertion sort бичихдээ шалгах 4 зүйл

1. `key = items[i]` — оруулах элементийг шилжүүлэхээс **өмнө** хадгалсан уу?
2. `j >= 0` шалгалт `items[j]`-ээс **өмнө** байна уу?
3. `while` (if биш), `j -= 1` давталт дотор байна уу?
4. key-г `items[j + 1]`-д (items[j] биш) оруулж байна уу?

Санаж үлдэх зүйлс

- Зүүн хэсэг эрэмбэлэгдсэн боловч эцсийн биш; key-ээс их элементүүд баруун тийш **шилжинэ**.
- Ажил өгөгдлөөс хамаарна: эрэмбэлэгдсэн — **O(n)**, урвуу — **O(n²)**.
- Тогтвортой: ижил элементүүдийн дараалал хадгалагдана (`>`, `>=` биш).
- Жижиг эсвэл бараг эрэмбэлэгдсэн өгөгдөлд — insertion sort; том өгөгдөлд — O(n log n) (3/3).